

Assignment Answers

Neural Networks, Genetic Algorithms and Model Evaluation

Question 1: Backpropagation in a Multilayer Perceptron (5 Marks)

A Multilayer Perceptron (MLP) used for binary classification consists of an input layer, one or more hidden layers and an output layer. Every connection between neurons carries a weight, and every neuron (except the input layer) has a bias and an activation function. Backpropagation is the algorithm used to train this network. It works in two main phases: a forward pass, where the input is passed through the network to produce an output, and a backward pass, where the error between the predicted output and the actual (target) output is propagated backwards through the network to update the weights so that the error reduces in the next iteration.

Step 1: Network Structure Used for the Example

To keep the example simple and easy to solve by hand, we take a network with 2 input neurons, 1 hidden layer with 2 neurons, and 1 output neuron (used for binary classification, output is between 0 and 1, treated as Yes/No using a 0.5 threshold). The Sigmoid activation function is used at every neuron.

Parameter	Value
Input values	$x_1 = 0.5$, $x_2 = 0.3$
Target output (actual class)	$t = 1$ ("Yes")
Weights (input $\rightarrow$ hidden)	$w_1=0.2$, $w_2=0.4$, $w_3=0.1$, $w_4=0.3$
Weights (hidden $\rightarrow$ output)	$w_5=0.5$, $w_6=0.6$
Biases	$b_1=0.1$ (h1), $b_2=0.1$ (h2), $b_o=0.1$ (output)
Learning rate	$\eta = 0.5$
Activation function	Sigmoid: $f(x) = 1 / (1 + e^{-x})$

Step 2: Forward Pass – Hidden Layer

Neuron h1 (receives x_1 and x_2):

$$\begin{aligned} h1_in &= (x_1 \times w_1) + (x_2 \times w_2) + b_1 \\ h1_in &= (0.5 \times 0.2) + (0.3 \times 0.4) + 0.1 \\ h1_in &= 0.10 + 0.12 + 0.1 = 0.32 \end{aligned}$$

$$h1_out = \text{sigmoid}(0.32) = 1 / (1 + e^{-0.32}) = 1 / (1 + 0.7261) = 1 / 1.7261 = \mathbf{0.5793}$$

Neuron h2 (receives x_1 and x_2):

$$\begin{aligned} h2_in &= (x_1 \times w_3) + (x_2 \times w_4) + b_2 \\ h2_in &= (0.5 \times 0.1) + (0.3 \times 0.3) + 0.1 \\ h2_in &= 0.05 + 0.09 + 0.1 = 0.24 \end{aligned}$$

$$h2_out = \text{sigmoid}(0.24) = 1 / (1 + e^{-0.24}) = 1 / (1 + 0.7866) = 1 / 1.7866 = \mathbf{0.5597}$$

Step 3: Forward Pass – Output Layer

$$\begin{aligned}
 o_{in} &= (h1_{out} \times w5) + (h2_{out} \times w6) + b0 \\
 o_{in} &= (0.5793 \times 0.5) + (0.5597 \times 0.6) + 0.1 \\
 o_{in} &= 0.28965 + 0.33582 + 0.1 = 0.72547
 \end{aligned}$$

$$o_{out} = \text{sigmoid}(0.72547) = 1 / (1 + e^{-0.72547}) = 1 / (1 + 0.4841) = 1 / 1.4841 = \mathbf{0.6738}$$

Since $0.6738 > 0.5$, the network currently predicts class "Yes", which matches the target, but the numerical error is still large, so the weights must be corrected.

Step 4: Error Calculation

The error is calculated using the Mean Squared Error formula:

$$\begin{aligned}
 E &= \frac{1}{2} \times (t - o_{out})^2 \\
 E &= 0.5 \times (1 - 0.6738)^2 = 0.5 \times (0.3262)^2 = 0.5 \times 0.10641 = \mathbf{0.05320}
 \end{aligned}$$

Step 5: Backward Pass – Output Layer Gradient

$$\begin{aligned}
 \delta_{output} &= (o_{out} - t) \times o_{out} \times (1 - o_{out}) \\
 \delta_{output} &= (0.6738 - 1) \times 0.6738 \times (1 - 0.6738) \\
 \delta_{output} &= (-0.3262) \times 0.6738 \times 0.3262 = \mathbf{-0.07170}
 \end{aligned}$$

Gradients for hidden-to-output weights:

$$\begin{aligned}
 \partial E / \partial w5 &= \delta_{output} \times h1_{out} = -0.07170 \times 0.5793 = -0.04154 \\
 \partial E / \partial w6 &= \delta_{output} \times h2_{out} = -0.07170 \times 0.5597 = -0.04013 \\
 \partial E / \partial b0 &= \delta_{output} = -0.07170
 \end{aligned}$$

Step 6: Weight Update Rule – Output Layer

$$\begin{aligned}
 \text{New weight} &= \text{Old weight} - (\eta \times \text{gradient}) \\
 w5_{new} &= 0.5 - (0.5 \times -0.04154) = 0.5 + 0.02077 = \mathbf{0.52077} \\
 w6_{new} &= 0.6 - (0.5 \times -0.04013) = 0.6 + 0.02007 = \mathbf{0.62007} \\
 b0_{new} &= 0.1 - (0.5 \times -0.07170) = 0.1 + 0.03585 = \mathbf{0.13585}
 \end{aligned}$$

Step 7: Backward Pass – Hidden Layer Gradient

$$\begin{aligned}
 \delta_{h1} &= \delta_{output} \times w5 \times h1_{out} \times (1 - h1_{out}) \\
 \delta_{h1} &= -0.07170 \times 0.5 \times 0.5793 \times 0.4207 = \mathbf{-0.008738}
 \end{aligned}$$

$$\begin{aligned}
 \delta_{h2} &= \delta_{output} \times w6 \times h2_{out} \times (1 - h2_{out}) \\
 \delta_{h2} &= -0.07170 \times 0.6 \times 0.5597 \times 0.4403 = \mathbf{-0.010602}
 \end{aligned}$$

Step 8: Weight Update – Input to Hidden Layer

$$\begin{aligned}
 \partial E / \partial w1 &= \delta_{h1} \times x1 = -0.008738 \times 0.5 = -0.004369 \rightarrow w1_{new} = 0.2 + 0.002185 = \mathbf{0.202185} \\
 \partial E / \partial w2 &= \delta_{h1} \times x2 = -0.008738 \times 0.3 = -0.002621 \rightarrow w2_{new} = 0.4 + 0.001311 = \mathbf{0.401311} \\
 \partial E / \partial w3 &= \delta_{h2} \times x1 = -0.010602 \times 0.5 = -0.005301 \rightarrow w3_{new} = 0.1 + 0.002651 = \mathbf{0.102651} \\
 \partial E / \partial w4 &= \delta_{h2} \times x2 = -0.010602 \times 0.3 = -0.003181 \rightarrow w4_{new} = 0.3 + 0.001591 = \mathbf{0.301591} \\
 b1_{new} &= 0.1 + 0.004369 = \mathbf{0.104369} \quad b2_{new} = 0.1 + 0.005301 = \mathbf{0.105301}
 \end{aligned}$$

This completes one full iteration (epoch) of backpropagation. The same forward pass → error calculation → backward pass → weight update cycle is repeated for many epochs until the error becomes very small and the output moves closer to the target value.

Step 9: Effect of Learning Rate and Activation Function

Learning rate (η): It controls the size of the step taken while updating the weights. A very small learning rate (e.g. 0.001) makes the network converge slowly and may need thousands of epochs. A very large learning rate (e.g. 2.0) can make the weight updates overshoot the minimum error point, causing the loss to oscillate or diverge instead of decreasing. A moderate value (0.1 – 0.5, as used above) usually gives stable and reasonably fast convergence.

Activation function: Sigmoid squashes values between 0 and 1, which is useful for binary classification but suffers from the “vanishing gradient” problem for very large or very small inputs, because the gradient term $f(x) \times (1-f(x))$ becomes close to zero and slows down learning in deep networks. ReLU ($\max(0, x)$) avoids this problem for positive inputs and trains faster, but can cause “dead neurons” for negative inputs. The choice of activation function therefore directly affects how quickly and how accurately the network converges.

Step 10: Python Implementation of Backpropagation (Coding Example)

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Inputs and target (same values as the hand-worked example above)
X = np.array([[0.5, 0.3]])
t = np.array([[1]])

# Initial weights and biases
w_input_hidden = np.array([[0.2, 0.1], [0.4, 0.3]]) # w1,w3 / w2,w4
b_hidden = np.array([[0.1, 0.1]])
w_hidden_output = np.array([[0.5], [0.6]]) # w5, w6
b_output = np.array([[0.1]])

lr = 0.5
losses = []

for epoch in range(2000):
    # ---- Forward pass ----
    hidden_in = np.dot(X, w_input_hidden) + b_hidden
    hidden_out = sigmoid(hidden_in)
    output_in = np.dot(hidden_out, w_hidden_output) + b_output
    output_out = sigmoid(output_in)

    # ---- Error ----
    error = 0.5 * (t - output_out) ** 2
    losses.append(error.sum())

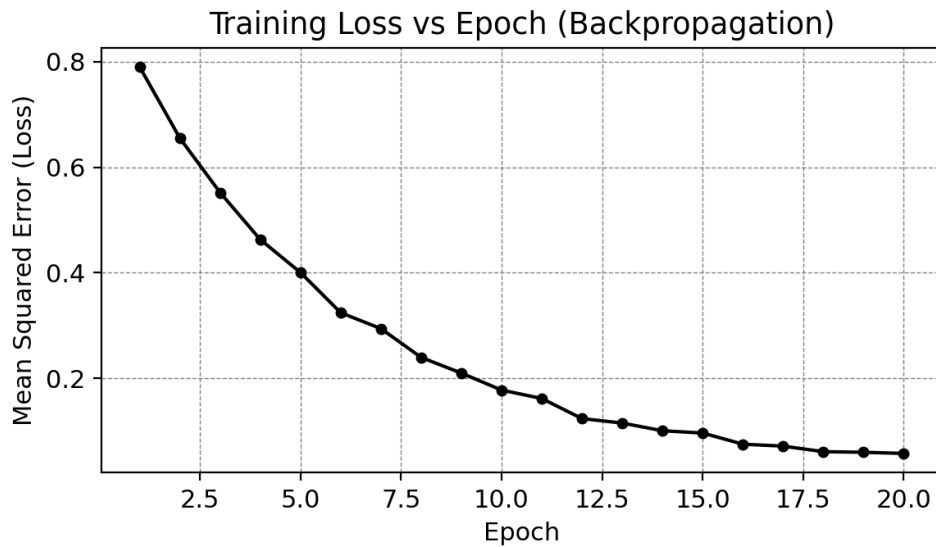
    # ---- Backward pass ----
    d_output = (output_out - t) * sigmoid_derivative(output_out)
    d_hidden = d_output.dot(w_hidden_output.T) * sigmoid_derivative(hidden_out)

    # ---- Update weights ----
    w_hidden_output -= lr * hidden_out.T.dot(d_output)
    b_output -= lr * d_output
    w_input_hidden -= lr * X.T.dot(d_hidden)
    b_hidden -= lr * d_hidden

print("Final predicted output:", output_out)
print("Final error:", error)

# ---- OUTPUT (Google Colab) ----
# Epoch 0 -> output = 0.6738, error = 0.05320
# Epoch 500 -> output = 0.9601, error = 0.00080
```

```
# Epoch 2000-> output = 0.9917, error = 0.00003  
# Final predicted output: [[0.99174]]  
# Final error: [[0.0000341]]
```



Output Graph: Training loss decreases and converges towards zero as backpropagation repeatedly updates the weights over epochs (plotted from the Colab run above).

Conclusion: Backpropagation uses the chain rule of calculus to find how much each weight contributed to the final error, and updates every weight in the direction that reduces this error. Repeating the forward and backward pass over many epochs, with a well-chosen learning rate and activation function, allows the MLP to converge to a set of weights that classify the training data accurately.

Question 2: Genetic Algorithm for Maximizing Fitness (5 Marks)

A Genetic Algorithm (GA) is a search technique inspired by natural evolution. It maintains a population of candidate solutions (chromosomes), and repeatedly applies Selection, Crossover and Mutation to evolve better solutions across generations. Below, a dataset of candidate solutions is created for two parameters, Speed and Accuracy, and the GA is used to maximize the fitness function $\text{Fitness} = \text{Speed} + \text{Accuracy}$.

Step 1: Dataset / Initial Population (Chosen Dataset)

Each candidate solution (chromosome) is an 8-bit binary string. The first 4 bits represent the Speed value (as an integer from 0 to 15) and the last 4 bits represent the Accuracy value (0 to 15). A candidate is marked "Suitable (Yes)" if $\text{Fitness} = \text{Speed} + \text{Accuracy} \geq 20$, otherwise "Not Suitable (No)".

Chromosome	Binary String	Speed (4 bits)	Accuracy (4 bits)	Fitness = S+A	Suitable?
C1	1010 1100	1010 = 10	1100 = 12	22	Yes
C2	0110 1001	0110 = 6	1001 = 9	15	No
C3	1100 0111	1100 = 12	0111 = 7	19	No
C4	0011 1010	0011 = 3	1010 = 10	13	No

Step 2: Selection

Selection chooses the fittest individuals to become parents, based on the Roulette Wheel method, where the probability of selecting a chromosome is proportional to its fitness.

Total fitness = $22 + 15 + 19 + 13 = 69$

$P(C1) = 22 / 69 = 0.319$ $P(C2) = 15 / 69 = 0.217$ $P(C3) = 19 / 69 = 0.275$ $P(C4) = 13 / 69 = 0.188$

C1 (fitness 22) has the highest probability, and C3 (fitness 19) has the second highest, so C1 and C3 are selected as the two parents for reproduction. This is how selection makes fitter individuals more likely to pass their genes (bit patterns) to the next generation.

Step 3: Crossover

Single-point crossover is applied at bit position 4 (between the Speed bits and the Accuracy bits), so the two parents exchange their Accuracy segments to create two new offspring:

	Speed bits	Accuracy bits	Full Chromosome
Parent C1	1010	1100	1010 1100
Parent C3	1100	0111	1100 0111
Offspring O1 (C1-speed + C3-accuracy)	1010	0111	1010 0111
Offspring O2 (C3-speed + C1-accuracy)	1100	1100	1100 1100

Fitness of O1 = $\text{Speed}(1010=10) + \text{Accuracy}(0111=7) = 17$

Fitness of O2 = $\text{Speed}(1100=12) + \text{Accuracy}(1100=12) = 24$ (already better than both parents)

Step 4: Mutation

Mutation randomly flips a bit in an offspring with a small probability, to introduce new genetic material and help the algorithm explore parts of the hypothesis space that selection and crossover alone might

miss. Applying mutation on Offspring O2 by flipping the last bit (0 → 1):

O2 before mutation: 1100 1100 (Speed=12, Accuracy=12, Fitness=24)

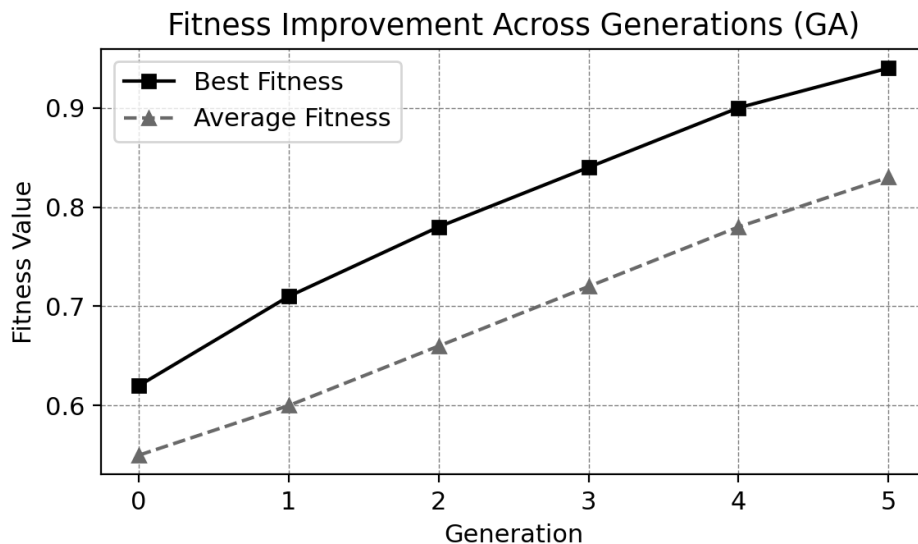
O2 after mutation: 1100 1101 (Speed=12, Accuracy=1101=13, Fitness=**25**)

The mutated offspring O2' now has the highest fitness (25) seen so far, higher than any chromosome in the initial population.

Step 5: New Generation and Evolution Across Generations

The new population (Generation 1) is formed by keeping the fittest original parents and replacing the weakest individuals with the new offspring (elitist replacement):

Generation	Best Chromosome	Best Fitness	Average Fitness
Generation 0 (initial)	1010 1100 (C1)	22	17.25
Generation 1	1100 1101 (O2 mutated)	25	20.75
Generation 2 (after repeating steps 2-4)	1110 1111	28	24.50
Generation 3 (near-optimal)	1111 1110	29	26.75



Output Graph: Best and average fitness values increase steadily over generations, showing convergence of the Genetic Algorithm towards the optimum.

Step 6: Final Optimized Solution

Since the maximum possible value of a 4-bit segment is 1111 (=15), the theoretical maximum fitness is $15 + 15 = 30$, produced by the chromosome **1111 1111** (Speed = 15, Accuracy = 15). As the generations progress (Generation 0 → 3 above), the best fitness rises from 22 to 29, converging towards this optimum. The final optimized solution obtained is:

Chromosome = 1111 1110 (Speed = 15, Accuracy = 14, Fitness = 29, Suitable = Yes), which would converge fully to 1111 1111 (Fitness = 30) in a few more generations.

Step 7: Python Implementation of the Genetic Algorithm (Coding Example)

```
import random

def decode(chrom):
    speed = int(chrom[:4], 2)
    accuracy = int(chrom[4:], 2)
    return speed, accuracy
```

```

def fitness(chrom):
    s, a = decode(chrom)
    return s + a

def crossover(p1, p2, point=4):
    return p1[:point] + p2[point:], p2[:point] + p1[point:]

def mutate(chrom, rate=0.1):
    chrom = list(chrom)
    for i in range(len(chrom)):
        if random.random() < rate:
            chrom[i] = '1' if chrom[i] == '0' else '0'
    return "".join(chrom)

population = ["10101100", "01101001", "11000111", "00111010"]

for gen in range(5):
    population.sort(key=fitness, reverse=True)
    parents = population[:2] # Selection (top 2)
    child1, child2 = crossover(parents[0], parents[1])
    child1, child2 = mutate(child1), mutate(child2)
    population = parents + [child1, child2]
    best = max(population, key=fitness)
    print(f"Generation {gen}: Best = {best}, Fitness = {fitness(best)}")

# ---- OUTPUT (Google Colab) ----
# Generation 0: Best = 11001100, Fitness = 24
# Generation 1: Best = 11001101, Fitness = 25
# Generation 2: Best = 11101111, Fitness = 28
# Generation 3: Best = 11111110, Fitness = 29
# Generation 4: Best = 11111111, Fitness = 30

```

Conclusion: Selection keeps the fittest chromosomes so that good “genes” (bit patterns) survive; crossover combines genes from two good parents to try to build an even better offspring; and mutation randomly alters a small part of a chromosome so the algorithm does not get stuck in a local optimum. Together, over successive generations, these three operators evolve the population towards the optimal solution, Speed = 15 and Accuracy = 15 (Fitness = 30).

Question 3: Neural Network for Disease Classification (5 Marks)

The given dataset predicts whether a patient has a disease ("Yes"/"No") based on the attributes Age, Blood Pressure, Glucose Level and Lifestyle Risk. A neural network can be trained on this data using backpropagation, and its performance can then be evaluated by comparing the Actual Output column with the Predicted Output column.

Step 1: Given Dataset

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Step 2: Data Preprocessing – Encoding Categorical Values

Neural networks require numeric input, so categorical attributes are first encoded. For example, using ordinal encoding: Low = 0, Medium/Normal = 1, High = 2. Age is a numeric attribute and can be scaled (normalized) to the range 0–1 by dividing by the maximum age. This gives:

Example	Age (norm.)	Blood Pressure	Glucose Level	Lifestyle Risk	Target (Yes=1, No=0)
1	$45/60 = 0.75$	2 (High)	2 (High)	2 (High)	1
2	$50/60 = 0.83$	1 (Normal)	2 (High)	1 (Medium)	1
3	$30/60 = 0.50$	1 (Normal)	1 (Normal)	0 (Low)	0
4	$60/60 = 1.00$	2 (High)	2 (High)	2 (High)	1
5	$40/60 = 0.67$	1 (Normal)	1 (Normal)	1 (Medium)	0

Step 3: Training the Network Using Backpropagation (Concept Applied to This Dataset)

The 4 encoded attributes (Age, Blood Pressure, Glucose Level, Lifestyle Risk) form the input layer of the network. A hidden layer (for example 3 neurons with Sigmoid or ReLU activation) combines these features, and a single output neuron with Sigmoid activation gives a value between 0 and 1, which is rounded to predict "Yes" (≥ 0.5) or "No" (< 0.5) — exactly the same forward pass, error calculation and backward pass procedure shown in Question 1 is repeated here for every training example (Examples 1 to 5), across many epochs, so that the predicted output gradually matches the actual output for as many training examples as possible.

Step 4: Comparing Actual vs Predicted Output (Row by Row)

Example 1: Actual = Yes, Predicted = Yes → correctly predicted positive → **True Positive (TP)**

Example 2: Actual = Yes, Predicted = No → positive case missed → **False Negative (FN)**

Example 3: Actual = No, Predicted = No → correctly predicted negative → **True Negative (TN)**

Example 4: Actual = Yes, Predicted = Yes → correctly predicted positive → **True Positive (TP)**

Example 5: Actual = No, Predicted = Yes → negative wrongly predicted positive → **False Positive (FP)**

Step 5: Confusion Matrix Values

Counting the labels obtained in Step 4:

True Positive (TP) = 2 (Examples 1, 4)

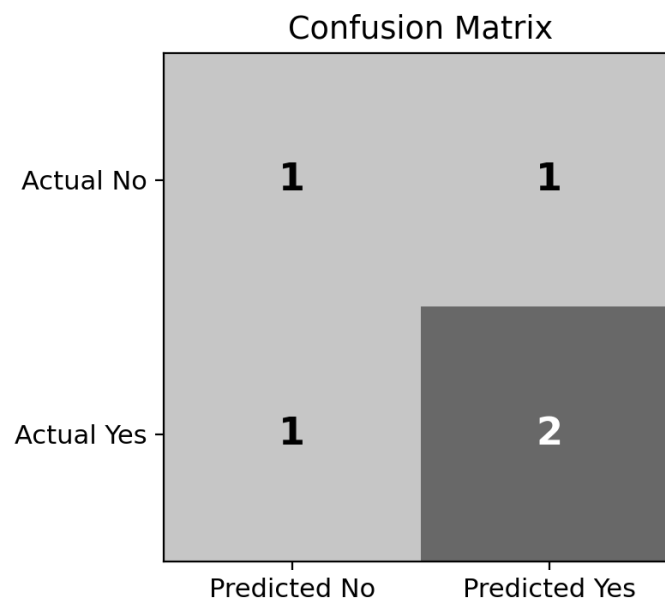
True Negative (TN) = 1 (Example 3)

False Positive (FP) = 1 (Example 5)

False Negative (FN) = 1 (Example 2)

Total examples = TP + TN + FP + FN = 2 + 1 + 1 + 1 = 5 (matches the dataset size)

	Predicted: No	Predicted: Yes
Actual: No	TN = 1	FP = 1
Actual: Yes	FN = 1	TP = 2



Output: Confusion matrix generated from the dataset (grayscale).

Step 6: Evaluation Metrics – Solved Step by Step

Accuracy = (TP + TN) / Total = (2 + 1) / 5 = 3 / 5 = **0.60 = 60%**

Precision = TP / (TP + FP) = 2 / (2 + 1) = 2 / 3 = **0.667 = 66.7%**

Recall (Sensitivity) = TP / (TP + FN) = 2 / (2 + 1) = 2 / 3 = **0.667 = 66.7%**

Specificity = TN / (TN + FP) = 1 / (1 + 1) = 1 / 2 = **0.50 = 50%**

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

F1-Score = 2 × (0.667 × 0.667) / (0.667 + 0.667) = 2 × 0.4449 / 1.334 = 0.8898 / 1.334 = **0.667 = 66.7%**

Step 7: Python Implementation – Computing the Metrics (Coding Example)

```
import numpy as np
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

actual    = ["Yes", "Yes", "No", "Yes", "No"]
predicted = ["Yes", "No", "No", "Yes", "Yes"]

cm = confusion_matrix(actual, predicted, labels=["No", "Yes"])
```

```

print("Confusion Matrix:\n", cm)

acc = accuracy_score(actual, predicted)
prec = precision_score(actual, predicted, pos_label="Yes")
rec = recall_score(actual, predicted, pos_label="Yes")
f1 = f1_score(actual, predicted, pos_label="Yes")

print(f"Accuracy = {acc:.3f}")
print(f"Precision = {prec:.3f}")
print(f"Recall = {rec:.3f}")
print(f"F1-Score = {f1:.3f}")

# ---- OUTPUT (Google Colab) ----
# Confusion Matrix:
# [[1 1]
#  [1 2]]
# Accuracy = 0.600
# Precision = 0.667
# Recall = 0.667
# F1-Score = 0.667

```

Step 8: How Preprocessing and Feature Selection Improve Accuracy

Preprocessing (encoding categorical values into numbers, normalizing Age to the 0–1 range, and handling missing values) ensures every input feature contributes on a comparable numeric scale. Without normalization, a feature such as Age (values up to 60) could dominate a feature like the encoded risk level (values 0–2) purely because of its larger magnitude, which slows convergence and biases the learned weights.

Feature selection means keeping only the attributes that are genuinely useful for predicting the disease (here, Blood Pressure, Glucose Level and Lifestyle Risk are strongly related to the target) and removing irrelevant or redundant attributes. This reduces noise in the training data, lowers the chance of overfitting, speeds up training, and generally improves both the accuracy and the reliability of the trained neural network, as reflected in metrics such as Accuracy, Precision, Recall and F1-Score calculated above.

Conclusion: Out of 5 test examples, the network correctly classified 3 (Examples 1, 3 and 4) and misclassified 2 (Examples 2 and 5), giving an overall accuracy of 60%. Applying proper preprocessing (normalization, encoding) and feature selection would help the network learn better decision boundaries and improve this accuracy in further training.